A D PATEL INSTITUTE OF TECHNOLOGY

THE CHARUTAR VIDHYA MANDAL UNIVERSITY

Name :- Jitiksha Patel

Enrolment Number :-12402080601054

Batch :- 1C8

Subject :- Java

Topic :- Assignment-2

1. Find prime numbers using multithreading (Thread,Runnable, Executor Framework)

```
import java.util.concurrent.*;

class PrimeUtil {
   static boolean isPrime(int n) {
      if (n < 2) return false;
      for (int i = 2; i <= n / 2; i++) {
         if (n % i == 0)
            return false;
      }
      return true;
   }
}

class PrimeThread extends Thread {
   int start, end;

   PrimeThread(int s, int e) {
      start = s;
      end = e;
   }

   public void run() {
      System.out.println("Thread Method:");
      for (int i = start; i <= end; i++) {
         if (PrimeUtil.isPrime(i))
            System.out.print(i + " ");
      }
      System.out.println("\n");
   }
}

class PrimeRunnable implements Runnable {
   int start, end;
   String label;

   PrimeRunnable(int s, int e, String label) {
      start = s;
      end = e;
      this.label = label;
   }
```

```
    public void run() {
        System.out.println(label);
        for (int i = start; i <= end; i++) {
            if (PrimeUtil.isPrime(i))
                System.out.print(i + " ");
        }
        System.out.println("\n");
    }
}

public class Program1 {
    public static void main(String[] args) throws Exception {

        int start = 1, end = 50;

        PrimeThread t1 = new PrimeThread(start, end);
        t1.start();
        t1.join();

        Thread t2 = new Thread(new PrimeRunnable(start, end, "Runnable Method:"));
        t2.start();
        t2.join();

        ExecutorService ex = Executors.newFixedThreadPool(1);
        ex.execute(new PrimeRunnable(start, end, "Executor Method:"));
        ex.shutdown();
    }
}
```

```
C:\Sem 4\Java\Assignment-2>java Program1
Thread Method:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Runnable Method:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Executor Method:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

2. Producer–Consumer problem using synchronization and inter-thread communication

```
class Buffer {
   int data;
   boolean available = false;

   synchronized void produce(int value) {
      try {
         while (available)
            wait();

         data = value;
         System.out.println("Produced: " + data);
         available = true;

         notify();
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }

   synchronized void consume() {
      try {
         while (!available)
            wait();

         System.out.println("Consumed: " + data);
         available = false;

         notify();
      } catch (InterruptedException e) {
         System.out.println(e);
      }
   }
}

class Producer extends Thread {
   Buffer b;

   Producer(Buffer b) {
      this.b = b;
   }
```

```
    public void run() {
        for (int i = 1; i <= 5; i++) {
            b.produce(i);
        }
    }
}

class Consumer extends Thread {
    Buffer b;

    Consumer(Buffer b) {
        this.b = b;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            b.consume();
        }
    }
}

public class Program2 {
    public static void main(String[] args) {
        Buffer b = new Buffer();

        Producer p = new Producer(b);
        Consumer c = new Consumer(b);

        p.start();
        c.start();
    }
}
```

```
C:\Sem 4\Java\Assignment-2>java Program2
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

3. Implement CRUD operations using Collection API (ArrayList, HashMap, TreeMap)

```
import java.util.*;

class Program3 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      ArrayList<String> list = new ArrayList<>();
      HashMap<Integer, String> map = new HashMap<>();
      TreeMap<Integer, String> tmap = new TreeMap<>();

      int choice;

      do {
         System.out.println("\n1.Create  2.Read  3.Update  4.Delete  5.Exit");
         System.out.print("Enter choice: ");
         choice = sc.nextInt();

         switch (choice) {

            case 1:
               System.out.print("Enter ID: ");
               int id = sc.nextInt();
               sc.nextLine();
               System.out.print("Enter Name: ");
               String name = sc.nextLine();

               list.add(name);
               map.put(id, name);
               tmap.put(id, name);

               System.out.println("Data Added");
               break;

            case 2:
               System.out.println("ArrayList: " + list);
               System.out.println("HashMap: " + map);
               System.out.println("TreeMap: " + tmap);
               break;

            case 3:
```

```
            System.out.print("Enter ID to update: ");
            int uid = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter new Name: ");
            String newName = sc.nextLine();

            if (map.containsKey(uid)) {
                map.put(uid, newName);
                tmap.put(uid, newName);
                System.out.println("Updated in Map");
            } else {
                System.out.println("ID not found");
            }

            break;

        case 4:
            System.out.print("Enter ID to delete: ");
            int did = sc.nextInt();

            map.remove(did);
            tmap.remove(did);

            System.out.println("Deleted from Map");
            break;

        case 5:
            System.out.println("Exit");
            break;

        default:
            System.out.println("Invalid choice");
    }

    } while (choice != 5);
  }
}
```

```
1.Create  2.Read  3.Update  4.Delete  5.Exit
Enter choice: 1
Enter ID: 1
Enter Name: xyz
Data Added

1.Create  2.Read  3.Update  4.Delete  5.Exit
Enter choice: 2
ArrayList: [xyz]
HashMap: {1=xyz}
TreeMap: {1=xyz}

1.Create  2.Read  3.Update  4.Delete  5.Exit
Enter choice: 3
Enter ID to update: 1
Enter new Name: abc
Updated in Map

1.Create  2.Read  3.Update  4.Delete  5.Exit
Enter choice: 4
Enter ID to delete: 1
Deleted from Map
```

4. Sort Book objects using Comparable and Comparator interfaces

```
import java.util.*;

class Book implements Comparable<Book> {
   int id;
   String name;
   double price;

   Book(int id, String name, double price) {
      this.id = id;
      this.name = name;
      this.price = price;
   }

   public int compareTo(Book b) {
      return this.id - b.id;
   }

   public String toString() {
      return id + " " + name + " " + price;
   }
}

class NameComparator implements Comparator<Book> {
   public int compare(Book b1, Book b2) {
      return b1.name.compareTo(b2.name);
   }
}

class PriceComparator implements Comparator<Book> {
   public int compare(Book b1, Book b2) {
      return Double.compare(b1.price, b2.price);
   }
}

public class Program4 {
   public static void main(String[] args) {

      ArrayList<Book> list = new ArrayList<>();

      list.add(new Book(3, "Java", 500));
      list.add(new Book(1, "Python", 400));
```

```
    list.add(new Book(2, "C++", 300));

    Collections.sort(list);
    System.out.println("Sorted by ID:");
    for (Book b : list)
      System.out.println(b);

    Collections.sort(list, new NameComparator());
    System.out.println("\nSorted by Name:");
    for (Book b : list)
      System.out.println(b);

    Collections.sort(list, new PriceComparator());
    System.out.println("\nSorted by Price:");
    for (Book b : list)
      System.out.println(b);
  }
}
```

```
C:\Sem 4\Java\Assignment-2>java Program4
Sorted by ID:
1 Python 400.0
2 C++ 300.0
3 Java 500.0

Sorted by Name:
2 C++ 300.0
3 Java 500.0
1 Python 400.0

Sorted by Price:
2 C++ 300.0
1 Python 400.0
3 Java 500.0
```

5. Count word occurrences from a βile using File Handling APIs

```
import java.io.*;
import java.util.*;

class Program5 {
  public static void main(String[] args) {
    HashMap<String, Integer> map = new HashMap<>();

    try {
      BufferedReader br = new BufferedReader(new FileReader("input.txt"));
      String line;

      while ((line = br.readLine()) != null) {
        String[] words = line.toLowerCase().split("\\W+");

        for (String word : words) {
          if (word.length() > 0) {
            map.put(word, map.getOrDefault(word, 0) + 1);
          }
        }
      }

      br.close();

      System.out.println("Word Occurrences:");
      for (Map.Entry<String, Integer> entry : map.entrySet()) {
        System.out.println(entry.getKey() + " : " + entry.getValue());
      }

    } catch (IOException e) {
      System.out.println("Error: " + e.getMessage());
    }
  }
}
```

```
C:\Sem 4\Java\Assignment-2>java Program5
Word Occurrences:
world : 1
hello : 1
```

6. Display all βiles of a given directory using File class

```
import java.io.File;
import java.util.Scanner;

class Program6 {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter directory path: ");
    String path = sc.nextLine();

    File dir = new File(path);

    if (dir.exists() && dir.isDirectory()) {
      String[] files = dir.list();

      System.out.println("Files in directory:");
      for (String f : files) {
        System.out.println(f);
      }
    } else {
      System.out.println("Invalid directory path");
    }
  }
}
```

```
C:\Sem 4\Java\Assignment-2>java Program6
Enter directory path: C:\Sem 4\Java\Assignment-2
Files in directory:
Book.class
Buffer.class
Consumer.class
input.txt
NameComparator.class
PriceComparator.class
PrimeRunnable.class
PrimeThread.class
PrimeUtil.class
Producer.class
Program1.class
Program1.java
Program1.java.bak
Program2.class
Program2.java
Program3.class
Program3.java
Program4.class
Program4.java
Program5.class
Program5.java
Program6.class
Program6.java
```

7. Implement TCP Echo Client–Server program

EchoServer

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

class EchoClient {
  public static void main(String[] args) {
    try {
      Socket socket = new Socket("localhost", 5000);

      BufferedReader in = new BufferedReader(
          new InputStreamReader(socket.getInputStream()));
      PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

      Scanner sc = new Scanner(System.in);
      String msg;

      while (true) {
        System.out.print("Enter message: ");
        msg = sc.nextLine();

        out.println(msg);

        String response = in.readLine();
        System.out.println("Server: " + response);

        if (msg.equalsIgnoreCase("exit"))
          break;
      }

      socket.close();

    } catch (Exception e) {
      System.out.println(e);
    }
  }
}
```

EchoClient

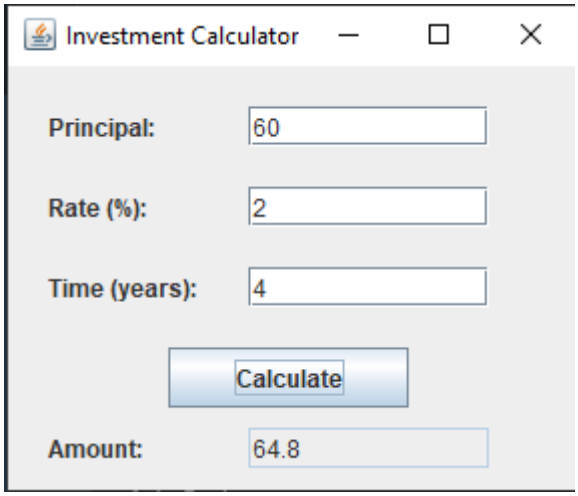
```
import java.io.*;
import java.net.*;
import java.util.Scanner;

class EchoClient {
  public static void main(String[] args) {
    try {
      Socket socket = new Socket("localhost", 5000);

      BufferedReader in = new BufferedReader(
          new InputStreamReader(socket.getInputStream()));
      PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

      Scanner sc = new Scanner(System.in);
      String msg;

      while (true) {
        System.out.print("Enter message: ");
        msg = sc.nextLine();

        out.println(msg);

        String response = in.readLine();
        System.out.println("Server: " + response);

        if (msg.equalsIgnoreCase("exit"))
          break;
      }

      socket.close();

    } catch (Exception e) {
      System.out.println(e);
    }
  }
}
```

```
C:\Sem 4\Java\Assignment-2>javac EchoServer.java

C:\Sem 4\Java\Assignment-2>javac EchoClient.java

C:\Sem 4\Java\Assignment-2>java EchoServer
Server started... Waiting for client
Client connected
Client: hello world!
Client: bye world!
Client: exit
```

```
C:\Sem 4\Java\Assignment-2>java EchoClient
Enter message: hello world!
Server: Echo: hello world!
Enter message: bye world!
Server: Echo: bye world!
Enter message: exit
Server: Echo: exit

C:\Sem 4\Java\Assignment-2>
```

8. Develop GUI-based Investment Calculator using Swing

```
import javax.swing.*;
import java.awt.event.*;

class Program8 extends JFrame implements ActionListener {

  JTextField principalField, rateField, timeField, resultField;
  JButton calculateBtn;

  Program8() {
    setTitle("Investment Calculator");
    setSize(300, 250);
    setLayout(null);

    JLabel l1 = new JLabel("Principal:");
    l1.setBounds(20, 20, 100, 20);
    add(l1);

    principalField = new JTextField();
    principalField.setBounds(120, 20, 120, 20);
    add(principalField);

    JLabel l2 = new JLabel("Rate (%):");
    l2.setBounds(20, 60, 100, 20);
    add(l2);

    rateField = new JTextField();
    rateField.setBounds(120, 60, 120, 20);
    add(rateField);

    JLabel l3 = new JLabel("Time (years):");
    l3.setBounds(20, 100, 100, 20);
    add(l3);

    timeField = new JTextField();
    timeField.setBounds(120, 100, 120, 20);
    add(timeField);

    calculateBtn = new JButton("Calculate");
    calculateBtn.setBounds(80, 140, 120, 30);
    add(calculateBtn);
```

```
        JLabel l4 = new JLabel("Amount:");
        l4.setBounds(20, 180, 100, 20);
        add(l4);

        resultField = new JTextField();
        resultField.setBounds(120, 180, 120, 20);
        resultField.setEditable(false);
        add(resultField);

        calculateBtn.addActionListener(this);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        try {
            double p = Double.parseDouble(principalField.getText());
            double r = Double.parseDouble(rateField.getText());
            double t = Double.parseDouble(timeField.getText());

            double amount = p + (p * r * t) / 100;

            resultField.setText(String.valueOf(amount));
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Invalid Input");
        }
    }

    public static void main(String[] args) {
        new Program8();
    }
}
```

```
C:\Sem 4\Java\Assignment-2>javac Program8.java

C:\Sem 4\Java\Assignment-2>java Program8
```